Faculty of Engineering and Technology
Electrical and Computer Engineering Department

Applied Cryptography
ENCS4320

HW 1

Partners:

Lara Foqaha	1220071	Section 3
Maysa Khanfar	1221623	Section 1

Date:
December, 2025

Q1) Many-Time Pad Attack

- **Problem Description**

  In this question, we analyzed multiple ciphertexts that were all encrypted using the same 8 bit one time pad (OTP) key. Reusing an OTP key is insecure because XORing two ciphertexts removes the key:

  $$Ci\ XOR\ Cj = Mi\ XOR\ Mj$$

  This exposes relationships between the plaintexts instead of hiding them. Our goal was to use these XOR relations to compute all possible plaintext characters at each byte position.

- **Method**

  Our approach used a combination of ASCII-based XOR classification and a constrained search over possible plaintext characters. We also assumed that the plaintext is either space, lowercase letter, or uppercase letter.

  1. **XOR Classification of Ciphertext Pairs**

     For each ciphertext set (C1, C2, C3) we computed the pairwise XORs:

     - C1 XOR C2
     - C1 XOR C3
     - C2 XOR C3

     Each XOR result is a single byte. We extracted the *upper 3 bits* of the XOR result, because these bits determine the ASCII category of the pair:

     - 000 → both plaintexts are same case (both uppercase or both lowercase)
     - 001 → one uppercase and one lowercase
     - 011 → uppercase with space
     - anything else → impossible under our assumption (plaintext ∈ {A-Z, a-z, space}).

     This allowed us to list all candidate plaintext type pairs (example: “uppercase / lowercase”, “space / lowercase”, etc.) for each XOR relation.

  2. **Space Character Deduction Using XOR Patterns**

     Because XOR with the space character (0x20) flips only the upper bits while preserving the lower bits, XOR categories involving "lower - space" or "upper - space" reveal that one of the two plaintexts must be a space. We counted how many XOR relations suggested that each plaintext position (m1, m2, m3) could be space. If any plaintext appeared in two or more such relations, then that plaintext is forced to be space. This deduction allowed us to recover partial plaintext information even when full decoding was not possible under our assumption.

3. **Valid Plaintext Search (Constrained Brute-Force)**

   Even though the OTP key is unknown, a key guess K produces plaintext candidates:

   $$m1 = C1 \text{ XOR } K,\ m2 = C2 \text{ XOR } K,\ m3 = C3 \text{ XOR } K$$

   For each $K \in \{0,1,2,\ldots,255\}$, we kept only plaintext triples that matched our allowed character set: uppercase letters A-Z, lowercase letters a-z, and space.

4. **Identifying Unique Solutions**
   - If exactly one plaintext set matched all constraints, then the plaintexts were fully recovered.
   - If multiple sets were valid, plaintext recovery was not unique and we reported only the candidate types and any forced space deductions.
   - If no set matched, then the ciphertext set was impossible under the assumed alphabet.

- **Results**

  We tested the program on four different ciphertext sets to evaluate how well many-time pad analysis reveals plaintext information under the assumption {space, A-Z, a-z}.

  For example, for **set 1: [EF, A4, D3]**
  - The XOR classifications showed a combination of “lower - space”, “upper - lower”, and “upper - space”, which is consistent with having exactly one space in the plaintext.
  - The brute-force search found exactly one valid plaintext set inside the allowed alphabet.
  - The program successfully recovered the plaintext: m1 = ‘k’, m2 = space, m3 = ‘W’

```
--- C1 xor C2 ---
 XOR result = 01001011  --> lower - space
    possible: lowercase / space
    possible: space / lowercase

--- C1 xor C3 ---
 XOR result = 00111100  --> upper - lower
    possible: uppercase / lowercase
    possible: lowercase / uppercase

--- C2 xor C3 ---
 XOR result = 01110111  --> upper - space
    possible: uppercase / space
    possible: space / uppercase

XOR deduction:
m2 must be a space character based on XOR results.

Recovered plaintexts:
m1 = 'k'  binary: 01101011
m2 = ' '  binary: 00100000
m3 = 'W'  binary: 01010111
```

Another example, **set 2: [10, 30, 50]:** Two XOR relations involved “letter - space”, and the program correctly identified m3 must be a space, based purely on XOR results. However, no plaintext set consisting only of letters or space satisfied all XOR constraints. This set shows that XOR analysis can reveal partial structure (such as which plaintext is space), but full recovery may still fail under strict alphabet limits. The program output:

```
--- C1 xor C2 ---
 XOR result = 00100000  --> upper - lower
    possible: uppercase / lowercase
    possible: lowercase / uppercase

--- C1 xor C3 ---
 XOR result = 01000000  --> lower - space
    possible: lowercase / space
    possible: space / lowercase

--- C2 xor C3 ---
 XOR result = 01100000  --> upper - space
    possible: uppercase / space
    possible: space / uppercase

XOR deduction:
m3 must be a space character based on XOR results.

Plaintext cannot be uniquely recovered (candidates above are the result).
```

As for **set 3: [B4, F9, 40]**, At least one XOR result was classified as “impossible for (upper, lower, space)”, meaning it does not relate to any XOR between two allowed plaintext characters.

For **set 4: [14, 36, 0F]**, All XOR results matched case-based categories (“same case”, “upper - lower”), but none involved space. When testing all possible keys, the program found multiple valid plaintext sets, all consisting of uppercase and lowercase letters. Because more than one plaintext satisfies the constraints, the program printed: “Plaintext cannot be uniquely recovered”. This example shows that even when XOR patterns reveal case information, the exact plaintext letters are not always uniquely determined.

- **Conclusion**

  This experiment showed that reusing an OTP key exposes relationships between plaintexts through XOR. In some cases, the plaintext was uniquely recovered, while in others only partial information (such as the location of a space) could be deduced. Certain ciphertexts did not match the assumed alphabet, and some produced multiple valid plaintexts, making recovery unclear. Overall, the attack demonstrates that many-time pad reuse compromises confidentiality and can reveal full or partial plaintext depending on the structure of the XOR results.

Q2) Vigenere Known-Plaintext Recovery

- **Problem Description**

  In this question, we are given a short piece of plaintext (HAPPPY) and its corresponding ciphertext (MOGZDD). The encryption was done using the Vigenere cipher, and we are told that the key length is at most 6. Our task is to determine whether the encryption key can be uniquely recovered, and to compute all possible keys that are consistent with the given data.

- **Method**

  To recover the key, we used the fact that in the Vigenere cipher the encryption rule is:

  $$ki = (ci - mi) \bmod 26$$

  So first each plaintext and ciphertext letter was converted into a number from 0 to 25 (A=0, B=1, C=2, …), and then the shift for every position was computed. This gives one possible "full key" of length 6. Since the key length is not known exactly, we tested all possible key lengths from 1 to the upper bound (6).

  For a candidate key length *L*, the Vigenere cipher repeats its key every *L* characters, so the key index used for position *i* (the index of the current letter in the plaintext/ciphertext) is:

  $$Key\ position = i \bmod L$$

  For each L, we built a temporary key and checked if it's possible. If a conflict appeared (two different shifts required at the same key position), then that key length is not possible. If there were no conflicts, then L is a valid key length and the corresponding key was recorded.

- **Expected Results**

Plaintext	a	b	c	d	e	f	g	h	i	j	k	l	m	n	o	p	q	r	s	t	u	v	w	x	y	z
Position	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25

Plaintext = "HAPPPY" / ciphertext = "MOGZDD"

1) Converting plaintext/ciphertext letters to numbers

i	Plaintext/message letter	Mi (message position)	Ciphertext letter	Ci (ciphertext position)
0	H	7	M	12
1	A	0	O	14
2	P	15	G	6
3	P	15	Z	25
4	P	15	D	3
5	Y	24	D	3

2) **Computing the shifts**
ki = (ci - mi) mod 26

i	Calculation	Ki	Letter
0	(12 - 7) mod 26	5	F
1	(14 - 0) mod 26	14	O
2	(6 - 15) mod 26	17	R
3	(25 - 15) mod 26	10	K
4	(3 - 15) mod 26	14	O
5	(3 - 24) mod 26	5	F

So the "full" key derived directly from the 6 characters is: FORKOF

3) **Checking all possible key lengths L ≤ 6**
The Vigenere key repeats every L characters. So we check whether a shorter key can generate the same 6 character sequence F O R K O F.
- L = 1, 2, 3, 4 → Not possible
- L = 5 → possible with key = FORKO. Positions 0 - 4 are F O R K O, and position 5 (which maps to index 0 again) matches F.
- L = 6 → possible with key = FORKOF

Two different keys satisfy the known plaintext-ciphertext pair:
FORKO (length 5)
FORKOF (length 6)
Therefore, the key is **not uniquely recoverable.**

- **Code Results**

```
Plaintext: HAPPPY
Ciphertext: MOGZDD
Computed 6 letter key: FORKOF
Possible keys with length <= 6:
  FORKO : length =  5
  FORKOF : length =  6
Key is not uniquely determined.
```

- **Conclusion**
The results show that both a 5 letter key (FORKO) and a 6 letter key (FORKOF) are fully consistent with the data. Since more than one key length produces a valid solution, the encryption key cannot be uniquely recovered using the available information.

Q3) Practical Distinguishers for the Linear PRF and Derived PRP

- **Problem Description**

  In this question, we implemented a simple pseudorandom function (PRF) of the form:

  $$F(x) = (a \cdot x + b) \bmod 2^{16}$$

  where the key consists of two random 16-bit values. We then used this PRF as the round function inside a 3-round Feistel network to build a pseudorandom permutation (PRP). Finally, we designed a distinguishing experiment that compares these constructions to truly random functions and permutations.

- **Method**

  1. **The PRF:**

     We first created a very simple PRF defined as: $F(x) = (a \cdot x + b) \bmod 2^{16}$

     This function is linear, which makes it predictable. In particular, the value

     $$F(x+1) - F(x) = a \text{ (always)}$$

     This kind of fixed behavior should not appear in a truly random function.

  2. **The PRF inside a 3-round Feistel network**

     Next, we used this PRF as the round function in a Feistel construction with three rounds. The 16-bit input was split into two 8-bit halves, and each round applied the Feistel update: $(L, R) \leftarrow (R,\ L\ XOR\ F(R))$

     Because the underlying PRF is linear, the Feistel-based PRP also keeps noticeable structure and is not secure.

  3. **Two "Ideal" references**

     To compare our weak constructions against something secure, we also implemented:

     - A random function, which gives a fresh random 16-bit output for every input.

     - A random permutation, created by shuffling all 16-bit values.

     These serve as the baselines for the distinguishing experiments.

  4. **Strong PRF distinguisher**

     We checked the difference: $D(x) = F(x+1) - F(x)$

     - For our PRF → all $D(x)$values are identical (always = a).

     - For a random function → the differences are random.

     So if all differences match → it's the weak PRF.

     This distinguisher succeeds almost **100%**.

5. **Strong PRP distinguisher**

   We measured: $D(x) = Enc(x)\ XOR\ Enc(x+1)$

   - With a random permutation → these values are scattered and rarely repeat.

   - With our Feistel PRP → the same differences show up again and again.

   If one difference occurs many times, the distinguisher labels it as the weak PRP. This method works well, usually achieving around **70–95%** accuracy.

- **Results**

  After running the PRF and PRP distinguishing tests, the program produced the following success rates:

```
=== Running PRF test (our version) ===
our PRF distinguisher success: 1.0000

=== Running PRP test (our version) ===
our PRP distinguisher success: 0.9840
```

  The PRF distinguisher identified the linear PRF perfectly in all trials (100%). This happened because the PRF has a fixed difference F(x+1) - F(x), which makes it extremely easy to recognize.

  The PRP distinguisher also performed very well, reaching about 98.4% accuracy. Although the Feistel structure adds some mixing, the PRP still inherits the linear behavior of the round function, so repeated XOR patterns appear frequently, making the distinguisher's decision straightforward.

- **Conclusion**

  Both distinguishers succeeded with very high accuracy, which confirms that the simple linear PRF and the Feistel PRP derived from it are not secure. Their outputs contain predictable patterns that never occur in truly random functions or permutations. As a result, an observer can reliably tell them apart from real randomness, demonstrating the weakness of using linear functions as cryptographic PRFs or PRPs.

Q4) Testing Shannon Perfect Secrecy

- **Problem Description**

  In this question, we were asked to write a program that checks whether a given encryption scheme satisfies Shannon Perfect Secrecy.

  According to the definition of perfect secrecy, an encryption scheme is perfectly secret if observing the ciphertext does not change the probability distribution of the underlying message:

  $$Pr[M = m \,/\, C = c] = Pr[M = m]$$

  Equivalently, the ciphertext distribution must be identical for all messages:

  $$Pr[Enc_k(m) = c] = Pr[Enc_k(m') = c]$$

  To test this condition, the program takes: a message space size m_size, a key space size k_size, a user defined encryption rule Enc(k,m), and a matching decryption rule Dec(k,c). then computes the ciphertext distributions for every possible message.

- **Method**

  We defined the message space and key space as: M={0,1,2},K={0,1,2}, so m_size = 3 and k_size = 3.
  Then an encryption function that behaves like addition modulo 3 was selected: Enc(k,m) = (m+k) mod3, and its matching decryption rule: Dec(k,c) = (c-k) mod3.

  To test perfect secrecy, the program does the following for each message m:

  1. It encrypts m using every key k in the key space with the enc function.
  2. It counts how many times each ciphertext c appears.
  3. It converts these counts into probabilities, giving the distribution: Pr[C = c | M = m]

  After computing the ciphertext distribution for every message, the program checks whether all distributions are identical. If all of them match, then the encryption scheme is perfectly secret. If even one distribution differs, perfect secrecy fails.

- **Expected Results**

  Since the encryption rule is: c = (m+k) mod3, for any fixed message m, and for uniformly chosen keys k ∈ {0,1,2}, the ciphertext takes each value in {0,1,2} exactly once:

Message	Possible ciphertexts c	Probability of each c
0	0, 1, 2	1/3 each
1	0, 1, 2	1/3 each
2	0, 1, 2	1/3 each

The probability distribution [1/3, 1/3, 1/3] is the same for all messages. According to Shannon's definition, the scheme is perfectly secret because the ciphertext reveals no information about which message was encrypted.

However, when we tested the scheme again with k_size = 2 and kept the message space the same (M={0,1,2}), the ciphertext distributions changed. In this case, some ciphertext values appeared more often than others depending on the message, causing the distributions Pr[C=c|M=m] to become different for different messages. When the key space became smaller than the message space, the encryption rule no longer produced uniform ciphertext distributions, and the scheme stopped being perfectly secret.

- **Code Results**

  Perfect secrecy success when m_size = k_size = 3:

```
Ciphertext distributions given each message:

M = 0: [0.3333333333333333, 0.3333333333333333, 0.3333333333333333]
M = 1: [0.3333333333333333, 0.3333333333333333, 0.3333333333333333]
M = 2: [0.3333333333333333, 0.3333333333333333, 0.3333333333333333]

The scheme is perfectly secret.
```

  Perfect secrecy fails when m_size = 3 and k_size = 2:

```
Ciphertext distributions given each message:

M = 0: [0.5, 0.5, 0.0]
M = 1: [0.0, 0.5, 0.5]
M = 2: [0.5, 0.0, 0.5]

The scheme is not perfectly secret.
```

- **Conclusion**

  The results show that with the original settings, the ciphertext distributions for all messages were identical, so the scheme was perfectly secret. When we changed the parameters and used a smaller key space, the distributions became different, and the encryption was no longer perfectly secret. This shows that perfect secrecy depends on both the encryption rule and the relationship between the message and key spaces.